

Studio OS - Reference IA et technique

Table of Contents

AI Operating Reference

Mission du systeme

Studio OS est la source d'état partagée entre deux humains et leurs agents. Une IA doit l'utiliser pour comprendre le contexte avant d'agir, journaliser ses actions importantes et éviter les collisions.

Regles de comportement

1. Avant une modification substantielle, récupérer la tâche et l'état du projet.
2. Vérifier claims et conflits avant de modifier des ressources sensibles.
3. Respecter les décisions DEC-XXXX déjà validées.
4. Utiliser Graphify pour cibler les fichiers avant un balayage massif du repo.
5. Utiliser la mémoire partagée en lecture; n'écrire que si le rôle l'autorise.
6. Les sous-agents Qwen ne doivent pas écrire directement dans la mémoire partagée par défaut.
7. Tout travail IA substantiel doit produire un AIWorkLog.
8. Ne jamais inventer l'état d'un autre développeur; interroger Studio OS.
9. Ne jamais supposer un LAN commun ou un accès direct à l'autre machine.
10. Pour les gros fichiers, utiliser Studio Transfer; ne jamais les pousser via l'API applicative.

Hierarchie de contexte

Ordre de confiance: 1. Contrats et décisions valides. 2. État courant Studio OS. 3. Git local et GitHub. 4. Graphify local. 5. Mémoire projet/studio. 6. Hypothèses de l'agent.

Cycle de tâche IA

- load task
- load project state
- inspect decisions
- inspect conflicts/claims
- fetch targeted graph context
- execute work
- run tests
- record changed files/tests/result

- set review state
- emit relevant events

Interdictions

- Pas de modification silencieuse des contrats partages.
- Pas de creation d'un backend parallele.
- Pas de synchronisation de memoire privatee.
- Pas de verrou bloquant des fichiers.
- Pas de suppression automatique d'un transfert non expire sans politique explicite. # Regles par type d'agent

Claude orchestrateur

- Peut raisonner sur architecture et code complexe.
- Peut creer/modifier des taches selon permission.
- Peut proposer/crire decisions si autorise.
- Peut proposer des ecritures de memoire.
- Doit deleguer les taches mecaniques a Qwen lorsque pertinent.
- Doit laisser un AIWorkLog.

Qwen local

- Priorite: taches mecaniques, tests, migrations simples, generation repetable, documentation technique ciblee.
- Memoire partagee: lecture seule par default.
- Ne prend pas de decision architecturale globale sans validation.
- Retourne fichiers modifies, tests et limites.

Brainstormer / Graphify curator

- Maintient le graphe local a jour.
- Recherche les relations code/tache/decision.
- Peut proposer de la memoire, mais la promotion en memoire partagee doit etre controlee.
- Dedoublonne les connaissances.

Studio Producer

- Ne code normalement pas.
- Analyse priorites, blocages, conflits, capacite humaine/IA.
- Propose parallelisation et decomposition.
- Ne cree pas de travail destructif sans regle explicite.

Marketing agents

- Consomment uniquement les captures/markers/candidats autorises.
- N'uploadent pas automatiquement toutes les captures brutes.
- Lient toute production au projet/tache/source. # Bootstrap compact pour une IA

Utiliser ce texte comme contexte initial minimal:

Tu travailles dans Studio OS, plateforme centrale d'un studio de deux developpeurs a distance. Les deux PC sont sur des reseaux differents et ne se joignent jamais directement. Le VPS central expose API/MCP/Dashboard et un stockage objet MinIO/S3. Avant de modifier un projet, recupere la tache, l'etat projet, les claims, conflits et decisions. Utilise Graphify pour cibler les fichiers. Respects la memoire et les decisions existantes. Qwen est read-only sur la memoire partagee par default. Les soft locks avertissent mais ne bloquent pas Git. Journalise tout travail IA substantiel. Les gros fichiers transitent directement vers MinIO via URLs pre-signees et multipart, jamais via FastAPI. Les clients doivent supporter l'offline et la resynchronisation idempotente. Les contrats API/EVENT/AUTH/SYNC sont versionnes et ne doivent pas etre modifies unilateralement.

Verification avant action

- Ai-je une Task ID ?
- Ai-je le ProjectState actuel ?
- Ai-je verifie les ResourceClaims ?
- Existe-t-il une Decision pertinente ?
- Ai-je cible les fichiers via Graphify/Git ?
- Mon action necessite-t-elle une review humaine ?
- Dois-je creer un AIWorkLog ou un Event ? # Architecture technique

Topologie

Deux clients distants -> HTTPS/WSS -> VPS OVH.

VPS

- Caddy
- FastAPI API
- MCP server
- PostgreSQL
- MinIO/S3
- Dashboard
- Worker process(es)

Chaque PC

- Studio daemon
- Studio CLI
- Godot watcher
- Git watcher
- Graphify adapter
- Obsidian adapter
- Recording provider
- SQLite offline queue

Separation des responsabilites

Le serveur detient l'etat partage, les permissions, l'historique et les metadonnees de stockage. Le client detient le contexte local, l'acces au repo, au vault, au graphe et aux captures.

Flux temps reel

WebSocket ou SSE pour taches, claims, presence, builds, AI work et notifications. Fallback polling.

Donnees volumineuses

Les donnees volumineuses utilisent un StorageProvider compatible S3. L'API ne proxyfie pas les gros fichiers. # API Contract v1

Base: /api/v1

Conventions

- JSON UTF-8.
- IDs internes UUID.
- IDs lisibles possibles pour Task/Decision/Transfer.
- Idempotency-Key supporte sur creations rejouables.
- Pagination: limit, offset ou curseur selon endpoint.
- Dates ISO 8601 UTC.

Endpoints principaux

Projects

- GET /projects
- GET /projects/{project_id}
- GET /projects/{project_id}/state

Tasks

- GET /tasks
- POST /tasks
- GET /tasks/{id}
- PATCH /tasks/{id}
- POST /tasks/{id}/claim
- POST /tasks/{id}/release

Sessions

- POST /sessions
- PATCH /sessions/{id}/end
- GET /sessions

Claims

- GET /claims
- POST /claims
- POST /claims/{id}/renew
- DELETE /claims/{id}

Decisions

- GET /decisions
- POST /decisions

Agents and AI work

- GET /agents
- POST /ai-work
- PATCH /ai-work/{id}
- GET /ai-work

Heartbeats

- POST /heartbeats

Events

- POST /events
- GET /events

Transfers

- POST /transfers
- GET /transfers
- GET /transfers/{id}
- POST /transfers/{id}/upload/initiate
- POST /transfers/{id}/upload/complete
- POST /transfers/{id}/download-url
- DELETE /transfers/{id}

Realtime

/api/v1/stream fournit les changements autorisés pour l'utilisateur connecté. # Event Contract v1

Envelope

```
{  
  "event_id": "uuid",  
  "event_type": "task.started",  
  "project_id": "uuid",  
  "task_id": "uuid-or-null",  
  "machine_id": "uuid-or-null",  
  "actor_type": "user | agent | system",
```

```
"actor_id": "uuid",  
"client_timestamp": "ISO8601",  
"server_timestamp": "ISO8601",  
"payload": {},  
"schema_version": 1  
}
```

Types

project.created

task.created, task.started, task.updated, task.blocked, task.completed

session.started, session.ended

resource.claimed, resource.renewed, resource.released, resource.conflict

decision.proposed, decision.created

agent.started, agent.stopped

ai_work.started, ai_work.completed, ai_work.failed, ai_work.review_requested

git.commit, git.branch.changed, git.pr.opened, git.pr.merged

graph.updated, memory.proposed, memory.updated

godot.started, godot.stopped

recording.started, recording.finished, recording.marker.created

build.started, build.succeeded, build.failed

transfer.created, transfer.uploading, transfer.ready, transfer.downloaded, transfer.expired,
transfer.deleted

marketing.candidate.created, marketing.post.published

Compatibilite

Ajouter des champs est permis si les anciens clients peuvent les ignorer. Un changement incompatible exige une nouvelle version de schema. # Auth & Sync Contract

Identites

- User identity
- Machine identity
- Agent identity

Une machine possede son propre credential revocable. Les agents peuvent heriter d'un contexte machine mais doivent garder leur identite logique.

Roles minimum

admin, developer, agent, readonly.

Synchronisation

Chaque ecriture offline-safe transporte un UUID stable et, si appropriée, une Idempotency-Key. Le serveur garantit qu'un replay identique ne crée pas un doublon.

Heartbeat

Intervalle nominal: 30 s. Etat dérive de last_seen_at avec seuils configurables.

Conflits de mise à jour

Les objets mutables utilisent updated_at et idéalement une version entière. En cas de conflit, le client doit recevoir 409 avec la version serveur courante.

Offline

La queue locale SQLite conserve payload, event_id, tentative, prochaine tentative, statut et erreur. Backoff exponentiel borné. Les actions critiques non rejouables doivent être marquées explicitement. # Modèle de données v1

Entités principales

Studio, User, Machine, Agent, Project, MachineProjectConfig, Task, WorkSession, ResourceClaim, Decision, AIWorkLog, Event, Transfer, TransferPart(optional), Notification, Build, Recording, RecordingMarker, MarketingCandidate.

Relations clefs

- Project 1-N Task.
- Task 1-N WorkSession / Decision / AIWorkLog / Transfer / Event.
- Machine 1-N Heartbeats / sessions / agents.
- Transfer relie sender, recipient, project/task optionnels et object_key MinIO.

Transfer

Champs minimum: id, transfer_code, sender_user_id, recipient_user_id, project_id, task_id, category, filename, object_key, content_type, size_bytes, sha256, status, expires_at, created_at, uploaded_at, downloaded_at, deleted_at.

ResourceClaim

TTL obligatoire. Un claim expire n'est plus considéré actif. Un claim de dossier entre en conflit avec un claim de fichier descendant. # Studio Transfer / Storage Specification

Objectif

Permettre l'échange de fichiers petits ou très gros entre les deux développeurs, avec contexte, reprise, intégrité et expiration.

Architecture

Studio API gère autorisation et métadonnées. MinIO/S3 stocke les octets. Le client upload/download directement via URLs pré-signées.

Upload petit fichier

1. POST /transfers.
2. API crée Transfer.
3. API fournit URL pré-signée PUT.
4. Client upload.
5. Client appelle complète avec taille/hash.
6. Serveur vérifie l'objet et passe ready.

Multipart gros fichier

1. Initiate multipart.
2. Client découpe en chunks (recommande 64-128 MiB).
3. Chaque part est envoyée directement au storage.
4. Les ETag/parts sont persistés localement.
5. Reprise après coupure sans recommencer les parts terminées.
6. Complete multipart puis validation taille/hash.

Download

URL GET pré-signée courte. Support HTTP Range pour reprise.

Retention suggérée

- transfer temporaire: 7 jours.
- build: 30 jours.
- asset: manuel ou longue retention.
- recording brut: local par défaut.

Sécurité

Bucket privé, URLs signées 10-30 min, quotas, taille max, noms non fiables, pas de chemin utilisateur direct dans object_key. Object key génère: studio/{project}/{yyyy}/{mm}/{transfer_uuid}/{safe_name}.

UX

Dashboard drag/drop, progression, vitesse, ETA, pause/reprise, incoming/outgoing, suppression, contexte projet/tache. CLI studio transfer send/list/incoming/outgoing/download/cancel/delete.

Stockage permanent

Studio Storage peut distinguer temporary, build, asset, export, archive. Les objets permanents doivent avoir retention explicite et etre visibles dans une vue de quota. # MCP Contract

Outils de base

studio_get_projects studio_get_project_state studio_get_task studio_get_active_tasks
studio_create_task studio_update_task studio_claim_task studio_release_task
studio_get_resource_claims studio_claim_resource studio_release_resource studio_get_decisions
studio_add_decision studio_get_recent_changes studio_get_sessions
studio_get_t teammate_activity studio_start_session studio_end_session studio_log_ai_work
studio_get_ai_work studio_memory_search studio_memory_read studio_graph_query
studio_generate_context_package studio_emit_event

Studio Transfer via MCP

studio_create_transfer_metadata studio_get_transfers studio_get_transfer
studio_request_transfer_download

Un agent ne doit normalement pas envoyer lui-meme plusieurs Go via MCP. Le MCP fournit metadata/authorization; le client local effectue le transfert S3.

Format

Reponses compactes, champs utiles uniquement, filtres project, task, since, limit. Les erreurs doivent etre explicites et machine-readable. # Offline Sync

But

Permettre au developpeur de continuer a travailler si le VPS ou Internet devient temporairement inaccessible.

Queue locale

SQLite avec tables pending_events, pending_mutations, pending_markers, sync_state et multipart_uploads.

Regles

- Toutes les operations rejouables ont un UUID stable.
- Le serveur repond de maniere idempotente.
- Ordre preserve par session lorsque necessaire.

- Les timestamps client sont conserves mais le serveur ajoute son timestamp.
- Une operation rejetee definitivement passe en dead_letter et est visible.

Transfers

Le multipart state conserve upload_id, object_key, parts terminees et ETags. Une coupure peut reprendre l'upload. # Obsidian & Graphify Integration

Memoire

Trois niveaux: private, project, studio. Private n'est jamais synchronise automatiquement. Project/Studio peuvent etre proposes puis approuves.

MemoryProvider

search, read, propose, write_if_authorized, append_task_log, create_decision_note.

Qwen

Lecture seule sur project/studio par default.

Graphify

Graphify est local et n'a pas besoin d'etre copie sur le VPS.

GraphProvider

refresh_graph, query, relevant_files, dependencies, related_symbols.

Context Package

Le serveur compose contexte partage; le client complete avec Git, Graphify, fichiers et memoire locale. Manifest versionne et trace les sources. # Test & Acceptance Plan

Tests backend

Auth, permissions, heartbeats, offline detection, task lifecycle, claim TTL, overlap detection, decisions, event idempotency, AI work, ProjectState, MCP.

Tests transfer

- fichier 1 Ko.
- fichier 1 Go multipart.
- interruption a 50% puis reprise.
- URL signee expiree.
- mauvais hash.
- quota depasse.
- suppression et expiration.
- download avec Range.

Tests clients

Daemon restart, queue persistante, reconnection, Git/Godot watcher, CLI, Graphify/Obsidian adapters, recording markers.

Tests bout-en-bout

Deux machines simulees sur reseaux differents, une tache par machine, claims concurrents, AIWorkLog, transfert de fichier, coupure reseau, reprise, review et resume quotidien.

Definition de done globale

Un nouveau poste peut etre configure sans modifier le serveur. Les deux developpeurs travaillent simultanement sans LAN commun. Les agents peuvent charger le contexte via MCP. Les gros transferts reprennent apres coupure. Les donnees privees restent locales. Le systeme est sauvegardable et restaurable. # Prompt Bloc A - Cloud/Core

Tu construis le Cloud/Core de Studio OS. Deux developpeurs travaillent depuis deux reseaux differents. Le serveur central sera heberge sur VPS OVH. Aucun acces direct entre postes n'est requis.

Responsabilites: FastAPI, PostgreSQL, MCP, auth, users/machines/agents/projects/tasks/sessions/claims/decisions/events/AIWorkLog, ProjectState, conflict detection, realtime, idempotency, StorageProvider/MinIO, TransferService, signed URLs, multipart coordination, quotas/retention, Docker/Caddy, backups, tests et contrats.

Priorite absolue: produire et maintenir les contrats TECH/02_API_CONTRACT.md, TECH/03_EVENT_CONTRACT.md, TECH/04_AUTH_SYNC_CONTRACT.md et les schemas partages afin que Bloc B avance avec mocks.

Ne construis pas le dashboard, daemon local, watchers, Graphify/Obsidian local, recorder ou Producer UI.

Pour les gros fichiers: ne jamais proxyfier le contenu via FastAPI. Fournir metadata, autorisation et URLs S3/MinIO pre-signees. Supporter multipart, resume, hash et expiration.

Definition de done: docker compose lance Caddy/API/MCP/Postgres/MinIO; deux machines distantes peuvent publier heartbeat, partager taches/claims/decisions/events, les MCP tools fonctionnent, les transferts volumineux reprennent, et les sauvegardes sont documentees. # Prompt Bloc B - Local Clients / Experience

Tu construis les clients locaux et l'experience utilisateur de Studio OS. Le backend est fourni par Bloc A et accessible via HTTPS. Ne construis pas un backend concurrent.

Responsabilites: StudioApiClient, MockStudioApiClient, daemon local, config, token storage, SQLite offline queue, heartbeat, CLI, dashboard, realtime, Git/Godot watchers, resource claim UX, Obsidian/Graphify adapters, context package local, RecordingProvider, markers, AI Work Ledger UI, Review Queue, Studio Producer, notifications, daily timeline, GitHub client, worker

framework, TransferClient, multipart uploader/downloader, TransferQueue, drag/drop UI et CLI transfert.

Le client doit fonctionner si le serveur est temporairement indisponible. Il ne doit jamais supposer que l'autre PC est joignable. Pour les transferts, il demande au backend des URLs signees puis parle directement a MinIO/S3.

Definition de done: deux postes sur deux reseaux differents voient le meme etat, travaillent offline temporairement, resynchronisent, detectent Git/Godot, utilisent Graphify/Obsidian localement, echangent de gros fichiers resumables, et accedent aux workflows IA/review/producer.